

과제4

- ▶ **다음 프로그램을 강의지원시스템에 5/9(토) 23:00까지 제출**
 - ▶ 각 과제별 폴더에 source code만 저장해서 파일 하나로 압축하여 제출
 - ▶ 폴더명: 1.CompleteBinaryTree / 2.TreeCalculator / 3. BST
 - ▶ String 관련 class 이외 자료구조 class는 사용 금지
- ▶ **과제1은 완전이진트리의 배열 형태, 과제2-3는 레퍼런스 방식으로 Tree 구현 할 것**
 - ▶ getLeftChild()와 getRightChild()만 각 형태에 맞게 잘 구현하면,
같은 method로 두가지 형태의 트리를 모두 처리할 수 있음을 잘 생각해 볼 것
- ▶ **트리 출력 10점 : 상속을 통해 1~3번의 출력을 동일한 print()로 구현 가능**
 - ▶ 트리 출력을 본인이 직접 구현했는지, 다른 코드 참조했으면 comment로 어디서 참조했는지 반드시 기술.
 - 뒤 예제의 방식은 추가적인 postorder traversal을 통해
현재 노드가 얼마 너비의 width에 출력되어야 할지를 미리 결정하여 노드의 멤버 변수에 저장하여 사용한 예
 - 모든 정보는 2글자 크기로 제한하여 구현(7점)해도 되나, 다양한 길이 처리도 도전(10점)할 것을 권장.
- ▶ **트리 출력이 어려우면 각 문제에서 지정한 순회 순열만 출력**
- ▶ **도전과제: 공통 generic과 상속을 사용하여 구현**

1.CompleteBinaryTree

- ▶ 노드 개수를 입력받아 **complete binary tree**를 구성하여 출력하고, preorder, inorder, postorder, level order로 traversal한 결과를 출력하시오.
- ▶ tree representation에서 **배열 구현 방식만** 사용하시오.
 - ▶ class Tree가 배열을 멤버 변수, getLeftChild() getRightChild()를 메소드로 가지게
 - ▶ class Node 또는 left/right 자식을 저장하는 변수가 필요 없음을 명심
- ▶ 수업시간에 다른 recursive 함수의 형태로 반드시 구현하시오.
 - ▶ void preorder(int nodeNum) / int getLeft(int nodeNum) 등 으로 구성
- ▶ 각 노드에는 preorder로 순회한 순서의 정수를 저장하시오.
 - ▶ preorder로 값을 채우고, 36진수 대문자로 출력하시오.
 - 입력되는 개수의 최대값은 1295로 제한
 - $0 \sim 1295_{10} \rightarrow 00 \sim ZZ_{36}$ 으로 tree에서는 모두 2글자로 출력
 - Integer.toString의 진법 변환으로 출력하는 경우 감점
- ▶ level order에서는 반드시 최초 크기가 4인 circular queue를 사용
- ▶ level order 출력에서 배열을 그대로 출력하거나 한꺼번에 enqueue했다가 deque 하는 방식으로 구현하지 말 것

```
14
1
+-----+
2          9
+-----+ +-----+
3      6      A      D
+--+ +--+ +--+ +--+
4 5 7 8  B  C  E  $
preorder   : 1 2 3 4 5 6 7 8 9 A ...
inorder    : 4 3 5 2 7 6 8 1 B A ...
postorder  : 4 5 3 7 8 6 2 B C A ...
level order: 1 2 9 3 6 A D 4 5 7 ...
```

2.TreeCalculator

▶ 입력된 계산식에 대한 Tree 표현을 출력하시오

- ▶ 과제3의 계산기 과제의 postfix notation 수식의 계산과정을 응용하면 tree를 얻을 수 있음
 - ▶ 계산 과정과 유사하게, postfix의 각 피연산자를 노드에 저장해서 stack에 넣고,
연산자가 나타나면 stack에서 두 개의 노드를 pop하여 subtree를 구성하고, 해당 root를 다시 stack에 push.
이 과정을 반복하면 마지막에는 stack에 전체 계산식의 root가 남게 됨.
- ▶ 대상 연산자 : 단항연산자 - ~, 이항 산술 연산자 : * / % + -, 이항 시프트 연산자 << >> >>> ,
이항 비트 연산자 & | ^, 괄호연산자 () { } []
 - ▶ 우선순위와 결합법칙은 자바와 동일.
- ▶ 피연산자는 정수. 연산자와 피연산자 사이에는 반드시 공백 있음
 - ▶ 공백 없는 것 처리되면 추가 점수
- ▶ 적절한 class로 tree를 구성하여, 다음 페이지 예제와 같이 연산식과 계산 과정을 tree로 표현하시오.
- ▶ 만들어진 tree를 preorder travers하여,
입력된 중위표기에 대한 전위표기 계산식을 tree 출력 전에 먼저 출력하시오.

2. TreeCalculator

10 * (5 - { 8 >>> 2 } ^ 4) - ~ ~ - ~ 59

postorder : 10 5 8 2 <<< - 4 ^ * 59 ~ m ~ ~ -

preorder : - * 10 ^ - 5 >>> 8 2 4 ~ ~ m ~ 59

result : 10

-(10)

+-----+

*(70)

~(60)

+---+

+-----+

10 ^ (7)

~(-61) \$

+-----+

+-----+

-(3) 4

m(60) \$

+-->>>(2)

+-----+

5 +---+

~(-60) \$

8 2

+---+

59 \$

3.BST

- ▶ **입력 문자열에 대해서 BST를 구성하고 출력하시오.**
 - ▶ 입력된 순서에 따라 각기 다른 트리가 나오는 점 확인 할 것
 - ▶ child가 하나만 존재하는 경우 반대쪽 child에 대한 출력은 'x'로 표현
 - ▶ 2번 출력과 같은 print() 메소드를 사용하도록 상속을 사용할 것을 추천
 - ▶ 트리 출력이 불가능한 경우에는
preorder와 inorder 순열만을 출력할 것
 - ▶ Java String의 compareTo()를 그대로 적용하면 됨
 - ▶ ex. "X"에 대해 "A"는 왼쪽 "a"는 오른쪽임
 - ▶ quit이 들어오면 반복 종료
 - ▶ 입력되는 값의 개수 제한 없음.

```
Ho Ta An Go Sa
preorder   : Ho An Go Ta Sa
inorder    : An Go Ho Sa Ta
Ho
+-----+
An      Ta
+----+ +----+
$  Go  Sa  $

Ta Ho Go
preorder   : Ta Ho Go
inorder    : Go Ho Ta
Ta
+-----+
Ho      $
+----+ +----+
Go $  $  $

quit
```

추가 자료

- ▶ preorder로 traversal 한 형태로 출력해도 됨

- ▶ 아래 방식은 같은 level에 있는 다른 노드의 최대 길이를 파악하면 됨

```
Horowitz +-- Anderson-Freed +-- $
          |                      +-- Goodrich
          +----- Tamassia +-- Sahn i
                          +-- $
```

```
Ho +-- An +-- $
   |      +-- Go
   +-- Ta +-- Sa
        +-- $
```

- ▶ 아래 방식은 ancestor들의 최대 길이만 기억하면 됨

```
Horowitz +-- Anderson-Freed +-- $
          |                      +-- Goodrich
          +-- Tamassia +-- Sahn i
                          +-- $
```

- ▶ 어려우면 각 노드의 최대 길이를 고정하여 구현해도 됨.
- ▶ 이외 무한한 출력 방식이 가능하니 자기 나름의 출력 방식을 고민해 보는 것을 강추